

Relazione progetto JBlackJack

Il progetto consisteva nel realizzare il gioco di carte BlackJack utilizzando il pattern **MVC** insieme al pattern **Observer-Observable**. Il progetto si divide in tre cartelle principali:

- **Model**
- **View**
- **Controller**

Model

Nella cartella Model possiamo trovare le classi che rappresentano la logica del gioco, abbiamo:

- **Mazzo:** viene creato un mazzo composto da 6 mazzi di 52 carte e viene mischiato, la classe include alcuni metodi fondamentali come il mescolamento del mazzo e la pesca della carta in cima. Qui viene implementato il Singleton pattern, poiché può esistere un solo mazzo comune a tutti i giocatori per partita.
- **Carta:** rappresenta una singola carta con il suo seme e il suo valore, possiamo trovare i metodi get e set, il metodo **getAsset()** che servirà, come dice il nome, a prendere il path della carta tramite nome e simbolo di quest'ultima per poi visualizzarla nella View, poiché nella cartella assets le carte sono salvate come **nome-simbolo.png**.
- **Mano:** questa classe è composta da una lista di Carta e rappresenta la Mano di ogni Giocatore. Qui possiamo trovare l'uso degli stream, esattamente nei metodi:
 - **valoreManoDealer()**: dove viene fatta la somma di tutte le carte nella mano, tranne di quella coperta, nella mano del dealer
 - **valoreMano()**: uguale al metodo di prima ma fa la somma di tutte le carte
 - **toString()**: semplicemente fa il toString grazie all'uso combinato di StreamBuilder e degli streamPossiamo anche trovare metodi come **getMano()**, **svuotaMano()** e **aggiungiCartaMano()** dove quest'ultima prende come parametro un oggetto di tipo Carta.
- **Giocatore:** questa è la classe che gestisce il Giocatore, qui possiamo **trovare l'implementazione del pattern Observer-Observable** poiché Giocatore estende Observable, quindi ogni volta che il Giocatore chiamerà dei metodi che vanno a cambiare il suo stato verrà inviata una notifica alla View dove questa cambierà la sua visualizzazione. Nella classe Giocatore possiamo trovare i metodi che gestiscono le azioni disponibili, come richiedere una carta, stare, pulire la mano, se abbiamo sballato.
- **Bot e Dealer:** queste due classi estendono Giocatore e hanno un loro metodo che gestisce la loro IA, in particolare il Dealer ha una IA più complicata poiché deve gestire le sue scelte in base al valore delle carte degli altri giocatori. **(ATTENZIONE! Di solito il Dealer nelle partite di BlackJack, quando il valore della sua mano supera o è uguale a 17 non può più richiedere carte, questa regola però dipende da casino' a casino' e nel nostro caso non viene considerata)**
- **Profilo:** questa classe gestisce il profilo del Giocatore, quindi il livello Giocatore, le partite giocate, le partite vinte, le partite perse e quelle finite in pareggio, abbiamo anche due metodi dove uno serve per salvare il Profilo in un file .txt dove dentro ci saranno tutte le informazioni relative al Profilo e l'altro invece servirà a caricare il Profilo leggendo quel

file .txt creato alla creazione del Profilo. Anche questa classe estende Observable poiche' ogni volta che si sceglia l'avatar la View dovra' aggiornarsi per mostrare l'immagine. Qui viene implementato anche il Singleton pattern poiche' puo' esistere un solo Profilo per Giocatore.

- **Partita:** qui e' presente la logica della partita, quindi la logica della vittoria, la logica per iniziare una partita in base a quanti bot si e' scelto, la logica della pescata e dello stare e quella dell'iniziare una nuova partita.

View

Nella cartella View possiamo trovare tutte le classi responsabili della visualizzazione del gioco. Ogni menu' presenta uno sfondo e un titolo realizzati tramite un semplice programma in Python, oltre a una canzone di sottofondo. Quindi possiamo trovare:

- **MenuView:** questa classe rappresenta il Menu iniziale del gioco, dove possiamo trovare tre tasti:
 - **Gioca:** una volta cliccato ci aprira' la finestra per la scelta dei bot nella partita
 - **Crea Profilo/Statistiche:** se il profilo non e' stato creato allora il tasto verra' visualizzato come '**Crea Profilo**', se invece il profilo e' gia' stato creato allora il tasto verra' visualizzato come '**Statistiche**'
 - **Esci:** chiude il gioco
- **CreazioneProfiloView:** questa classe rappresenta la finestra per la creazione del Profilo, dopo aver cliccato 'Crea Profilo' nella MenuView, si aprira' una schermata dove potremo scegliere un'avatar e il nome del profilo, una volta che questi due sono stati scelti bastera' schiacciare il tasto 'Salva' (**ATTENZIONE! Il nome non potra' contenere alcuni caratteri e dovra' per forza essere inserito, quest'ultima cosa vale anche per l'avatar**) Questa classe implementa **Observer** poiche' ogni volta che verra' chiamato il metodo **setAvatar()** nel controller della **CreazioneProfiloView**, verra' aggiornata la visualizzazione dell'avatar
- **StatisticheView :** questa classe rappresenta la finestra delle statistiche del Giocatore, dopo aver cliccato su 'Statistiche' nella MenuView, si aprira' una schermata dove sara' presente l'avatar scelto precedentemente e il **toString()** del **Profilo** per la visualizzazione del nome e di tutte le informazioni relative come le partite vinte, perse, giocate, pareggiate e il livello del Giocatore.
- **SceltaBotView :** questa classe rappresenta la finestra per la scelta di quanti bot volere nella partita (escluso il Dealer), dopo aver cliccato su 'Gioca' nella MenuView, si avranno 4 bottoni da poter cliccare
 - **1 Bot,**
 - **2 Bot,**
 - **3 Bot,**
 - **Torna indietro**
- **AudioManager :** questa classe si occupa di mandare in output i suoni per ogni interazione che si ha nelle view, come ad esempio il rumore delle carte pescate, il rumore dei tasti schiacciati e le canzoni di sottofondo, **qui viene utilizzato il pattern Singleton.**
- **GameView :** questa e' la classe dove viene visualizzata la partita, in base a quanti bot si scelgono verranno rappresentate le loro rispettive mani in basso allo schermo insieme alla

nostra mano dove, sopra quest'ultime, ci sarà scritto il nome del Giocatore/Bot e il rispettivo valore, mentre sopra al centro avremo la mano del Dealer con su scritto il suo valore della mano (escludendo la carta coperta) e il mazzo sulla sua destra. Il Giocatore sotto alla schermata potrà effettuare tre scelte:

- **Pesca** : azionerà la logica della **'pescata'**,
- **Stai** : azionerà la logica dello **'stai'**,
- **Rigioca** : cliccabile solo alla fine della partita,
- **Menu Principale** : ci farà tornare al Menu

Ogni volta che il Giocatore sballa o clicca il tasto stai, partiranno le IA dei Bot e del Dealer e una volta che quest'ultimo finisce entrerà in azione il **metodo logicaVittoria()** che manderà a schermo il risultato della partita

Controller

Nella cartella controller abbiamo tutte le classi che gestiscono le interazioni dell'utente, quindi sono classi che vanno a collegare la View e il Model, abbiamo:

- **MenuController**: si occupa della logica dei tasti **'Gioca'**, **'Crea Profilo/Statistiche'** ed **'Esci'** presenti nella **MenuView**, e **gestisce la navigazione** tra il menu principale e le altre view come **CreazioneProfiloView**, **StatisticheView** e **SceltaBotView**
- **CreazioneProfiloController**: si occupa della logica dei tasti, verifica la validità del nome e gestisce la scelta dell'avatar presenti nella **CreazioneProfiloView**, e interagisce con la classe **Profilo** per il salvataggio dei dati.
- **StatisticheController**: si occupa della logica del tasto presente nella **StatisticheView**
- **SceltaBotController**: si occupa della logica dei tasti presenti nella **SceltaBotView**
- **GameController**: si occupa della logica dei tasti **'Stai'**, **'Pesca'**, **'Rigioca'** e **'Menu Principale'** presenti nella **GameView**.
- **JBlackJack** : è il main dove viene inizializzato il path del profilo, controlla se questo esiste, se è vero allora carica il profilo e lo passa alla **MenuView**, se falso crea comunque un profilo senza nome e lo passa alla **MenuView**.